

A Simple and Efficient Algorithm for Finding Minimum Spanning Tree Replacement Edges

David A. Bader, Paul Burkhardt

<http://jgaa.info/> vol. 26, no. 4, pp. 577-588 (2022)

Nagy Bence

2025. december 1.

Tartalomjegyzék

- 1 Áttekintés: minimális feszítőfa, csereél
- 2 A feladat felállása
- 3 Az algoritmus alapötlete
- 4 Az algoritmus pszeudokódja
- 5 Példa lefutás
- 6 Időkomplexitás bizonyítása
- 7 Legfontosabb él (Most Vital Edge)
- 8 Összegzés

Áttekintés: minimális feszítőfa, csereél

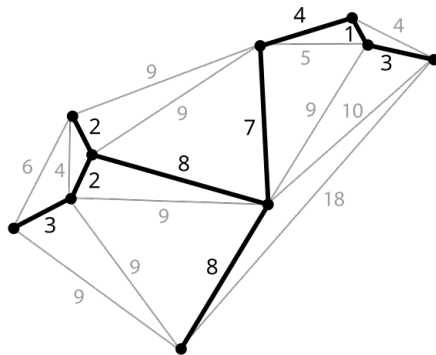
Definíció:

Legyen $G = (V, E)$ egy irányítatlan, súlyozott gráf, ahol $n = |V|$ és $m = |E|$, és minden $e \in E$ élhez tartozik egy $w(e)$ súly. Egy minimális feszítőfa $T = MST(G)$ a gráf olyan $n-1$ élből álló részhalmaza, amely összeköti az összes csúcsot, és az élek összsúlya minimális.

Definíció:

A csereél az a legkisebb súlyú él, amely újra összeköti a minimális feszítőfát egy minimális feszítőfabeli él törlése után.

A probléma alkalmazása: utakon balesetek, kommunikációs hálózatokban kábel szakadás esetén



Kép forrás: Wikipédia

- Adottak:
 - A gráf és a minimális feszítőfája (MST)
 - A nem feszítőfabeli élek súlyuk szerint **növekvő sorba rendezve**
- Keressük:
 - Csereéleket minden minimális feszítőfabeli élhez.

- Az algoritmus képes megtalálni minden csereélet, ha a nem min feszítőfabeli élek élsúly szerint növekvő sorrendben rendezve megvannak határozva:
 - **Idő- és tárbonyolultság:** $\mathcal{O}(m + n)$
(m : élek száma, n : csúcsok száma)
 - Megjegyzés az élek rendezettségére:
 - Ha az MST-t Kruskal algoritmussal kaptuk meg akkor rendezve vannak már a nem-MST élek.
 - Ha még nincsenek rendezve, akkor képesek vagyunk rendezni őket $\mathcal{O}(n)$ időben, ha ismerünk egy felsőkorlátot az élsúlyok értékeire.
- A cikk előtt lehetett tudni, hogy ez elméletileg lehetséges, de ez az első algoritmikus megoldás erre lineáris időkomplexitással.

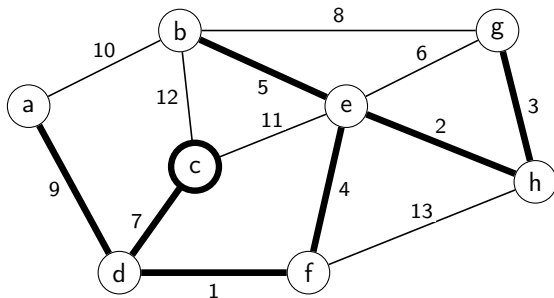
- 1975: Spira és Pan: MST egy éléhez csereél $\mathcal{O}(n^2)$ időben.
- 1978: Chin és Houck: MST minden éléhez csereél $\mathcal{O}(n^2)$ időben.
- 1979: Tarjan: $\mathcal{O}(m\alpha(m, n))$
 - α : Ackermann függvény inverze: nagyon-nagyon lassan nő a bemenethez képest.
 - pl: $\alpha(7) = 2$, $\alpha(61) = 3$, $\alpha(2^{2^{2^{2^{2^2}}}} - 3) = 4$
 - Majdnem lineáris megoldás, de mégsem az.
- 2022: Bader és Burkhardt: $\mathcal{O}(m + n)$

Az algoritmus alapötlete 1.

- Adott T és a nem-fa élek $E \setminus E_T$ súly szerint növekvő sorban.
- Figyeljük meg, hogy mindegyik $m - n + 1$ darab él $E \setminus E_T$ -ben egy fundamentális kört indukál T éleivel.

1. Állítás

Az a legkisebb súlyú nem-MST él, amely egy e MST-beli élt tartalmazó kört indukál, az az e él csereéle.



Az algoritmus alapötlete 2.

- 1 Kijelölünk egy tetszőleges csúcsot a fa gyökerének.
- 2 A mélységi bejárás alapján minden csúcshoz határozzuk meg a szülejét, valamint a belépési ($IN[v]$) és kilépési ($OUT[v]$) sorszámait.
- 3 Járjuk be a fundamentális kört, amelyet egy nem-fa él indukált.
- 4 Hajtsunk végre úttömörítés (path compression), hogy kihagyjuk azon élek bejárását, amelyenek már találtunk csereélet.

A diszjunkt halmaz adatszerkezetet használjuk gyors úttömörítésre a Gabow-Tarjan-féle statikus uniófa módszer alapján.

Definíció:

A diszjunkt halmaz adatstruktúra diszjunkt halmazok gyűjteményét tartja fenn:

$\mathcal{S} = \{S_1, S_2, \dots, S_k\}$. Minden halmazt egy **képviselőjével** azonosítjuk, amely tagja a halmaznak.

Műveleteket a diszjunkt halmaz adatstruktúrán:

- **makeSet(v)**: létrehoz egy új egy elemű halmazt: $\{v\}$, amelynek v lesz a képviselője.
- **find(v)**: visszaadja annak a diszjunkt halmaznak a képviselőjét, amelyben v van.
- **link(u, v)**: Létrehoz egy új halmazt, ami u és v halmazának az uniója.

Gabow és Tarjan algoritmusa a diszjunkt halmazok egy speciális esetére

- Speciális eset: a teljes uniófa (Union Tree) ismert az algoritmus indulásakor.
- Esetünkben az eredeti MST lesz a Union-Tree.
- Az algoritmus **m darab unió és find műveletet** hajt végre, **n darab elemen**: $\mathcal{O}(m+n)$ futási időben, $\mathcal{O}(n)$ tárhelykomplexitással
- Itt: $\text{link}(v)$: v halmazát és $P[v]$ halmazát összeuniózza és $P[v]$ képviselője lesz az új halmaz képviselője. ($P[v]$: v szülője a Union-Tree-ben)

Az algoritmus pszeudokódja (inicializációk és pathlabel meghívása)

```
24: Root the MST  $T$  at arbitrary vertex  $v_r$  and store parents in  $P$ .
25:  $P[v_r] \leftarrow v_r$  ▷ root's parent points to root
26: Run DFS on  $T$ , setting  $\text{IN}[v]$  and  $\text{OUT}[v]$  to the counter value when  $v$  is first and last visited,
    respectively.
27: for all vertices  $v \in V$  do
28:   makeset( $v$ ) ▷ Initialize the disjoint sets
29: for all edges  $e \in E_T$  do
30:    $R_e = \emptyset$  ▷ Initialize the replacement edges
31: for  $k \leftarrow 1 \dots m - n + 1$  do ▷ Scan the  $m - n + 1$  sorted non-MST edges
32:    $(v_i, v_j) \leftarrow e_k$ 
33:   PATHLABEL( $v_i, v_j, (v_i, v_j)$ )
34:   PATHLABEL( $v_j, v_i, (v_i, v_j)$ )
```

Az algoritmus pszeudokódja (PathLabel metódus)

```
1: procedure PATHLABEL( $s, t, e$ )
2:   if  $\text{IN}[s] < \text{IN}[t] < \text{OUT}[s]$  then                                ▷  $s$  is ancestor of  $t$ 
3:     return
4:   if  $\text{IN}[t] < \text{IN}[s] < \text{OUT}[t]$  then                                ▷  $t$  is ancestor of  $s$ 
5:      $\text{PLAN} \leftarrow \text{ANCESTOR}, k_1 \leftarrow \text{IN}[t], k_2 \leftarrow \text{IN}[s]$ 
6:   else
7:     if  $\text{IN}[s] < \text{IN}[t]$  then
8:        $\text{PLAN} \leftarrow \text{LEFT}, k_1 \leftarrow \text{OUT}[s], k_2 \leftarrow \text{IN}[t]$       ▷  $s$  is left of  $t$ 
9:     else
10:       $\text{PLAN} \leftarrow \text{RIGHT}, k_1 \leftarrow \text{OUT}[t], k_2 \leftarrow \text{IN}[s]$     ▷  $s$  is right of  $t$ 
11:    $v \leftarrow s$ 
12:   while  $k_1 < k_2$  do                                                ▷ Detecting when below LCA( $s, t$ )
13:     if  $\text{find}(v) = v$  then                                            ▷ If true, set replacement edge for  $(v, P[v])$ 
14:        $R_{(v, P[v])} \leftarrow e$                                        ▷ Set the replacement edge
15:       link( $v$ )                                                         ▷ Union the disjoint sets of  $v$  and  $P[v]$ 
16:    $v \leftarrow \text{find}(v)$ 
17:   switch  $\text{PLAN}$  do
18:     case  $\text{ANCESTOR}$ 
19:        $k_2 \leftarrow \text{IN}[v]$ 
20:     case  $\text{LEFT}$ 
21:        $k_1 \leftarrow \text{OUT}[v]$ 
22:     case  $\text{RIGHT}$ 
23:        $k_2 \leftarrow \text{IN}[v]$ 
```

- Meghatározza s és t élek pozícióját egymáshoz képest az MST-n. (LEFT, RIGHT, ANCESTOR)
- s éllel kezdve végig megyünk az $s \rightarrow t$ úton, és:
 - Ha az adott él még nem volt úttömörítve ($v = \text{find}(v)$), akkor annak az élnek az inputként megadott élet menti el csereélelnek. Majd úttömörítést hajtunk végre ($\text{link}(v)$).
 - k_1 és k_2 counter-eket léptetgeti, ameddig végig nem érünk az $s \rightarrow t$ úton vagy nem el nem érjük s és t legalacsonyabb közös őst.

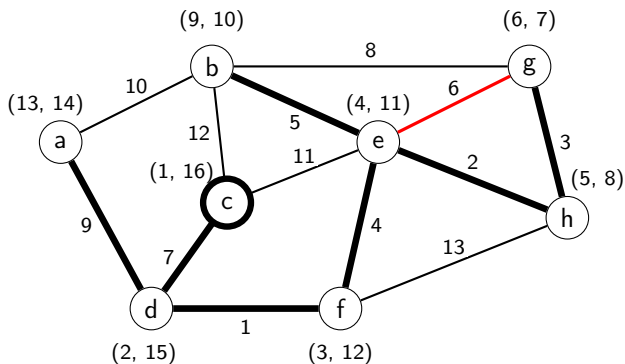
2. állítás

Az algoritmus a nem-MST él által indukált kört leszármazottól ősz felé járja be, és megáll a legalacsonyabb közös ősnél (abban az esetben, ha a legalacsonyabb közös ősz különbözik s -től és t -től)

Példa lefutás - első nem-fa él (e, g)

Első nem-fa él: (e, g)

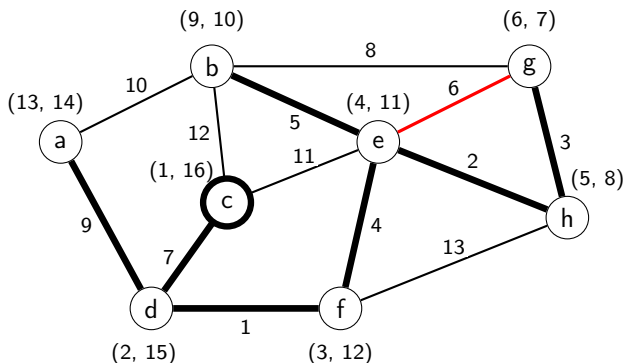
- Az e él g élnek az őse a fában, így megkapjuk $plan = ANCESTOR$ -t. Továbbá $k_1 = IN[e] = 4$ és $k_2 = IN[g] = 6$, és mivel $k_1 < k_2$ ezért elkezdjük a bejárást.
- A kör bejárás g -nél kezdjük. Mivel g -t még nem jártuk be ($link(g) = g$), ezért a (g, e) nem-MST élet (g, h) csereéleként mentjük el ($R(g, h) = (e, g)$).
- g és h diszjunkt halmazait link-eljük úgy, hogy g diszjunkt halmaza kapja meg h képviselőjét. Ez a lépés tömöríti a (g, h) részt.



Példa lefutás - első nem-fa él (e, g) folyt.

Első nem-fa él (e, g) : (folyt.)

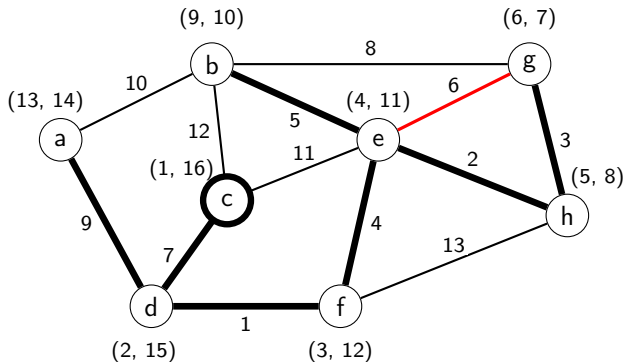
- A következő él a bejárásban h lesz, mivel g szülője ($\text{find}(g) = h$). k_2 -t frissítjük $\text{IN}[h]$ -vel. ($\text{IN}[h] = 5$)
- Mivel (e, h) él még nem lett tömörítve ($\text{find}(h) = h$), ezért (h, e) csereélenek (g, e) élt adjuk.
- h és e diszjunkt halmazait link-eljük úgy, hogy h diszjunkt halmaza kapja meg e képviselőjét. Ez tömöríti a (g, h, e) részutat.
- A bejárásban e lesz a következő él, szóval k_2 megkapja $\text{IN}[e]$ -t ($= 4$), már egyenlő lesz k_1 -el ($= 4$), ezzel leállítva a while ciklust.



Példa lefutás - első nem-fa él (e, g) folyt.

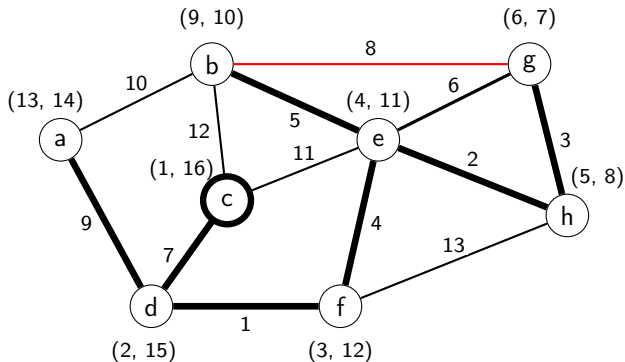
Első nem-fa él (e, g) : (folyt.)

- A fordított (e, g) éllel PathLabel hívás nem fog feldolgozódni, mivel e őse g -nek.



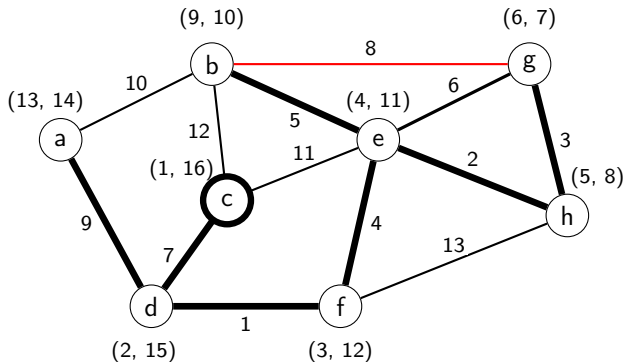
Második nem-fa él ($b-g$):

- Meghívjuk $\text{PathLabel}(b, g, (b, g))$ -t
- RIGHT plan-t kapjuk meg. $k_1 = \text{OUT}[g]$, $k_2 = \text{IN}[b]$, szóval $k_1 < k_2$.
- A bejárást b -vel kezdjük és mivel b -t még nem látogattuk meg ezért (b, e) él a (b, g) nem-MST élt kapja meg csereélként.
- b és e diszjunkt halmazait link-eljük. Ezzel tömörítjük a (b, e) részutat.



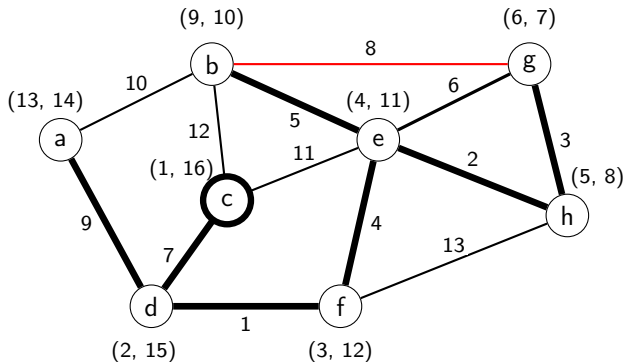
Második nem-fa él ($b-g$): (folyt.)

- A következő él a bejárásban e (mivel ő b-nek a szülője), így $k_2 = \text{IN}[e]$. Ekkor $k_2 < k_1$ teljesül, így leáll a while loop.
- Meghívjuk PathLabel-t ($g-b$) éllel: $\text{PathLabel}(g, b, (g, b))$.
- Megkapjuk LEFT ágtervet, $k_1 = \text{OUT}[g]$ és $k_2 = \text{IN}[b]$ -vel, szóval $k_1 < k_2$ és megkezdjük a bejárást g-vel kezdve.



Második nem-fa él ($b-g$): (folyt.)

- Itt figyeljük meg, hogy a diszjunkt halmazok tömörítették már a (g, h, e) részutat, ezért $find(g) \neq g$. Ekkor a e -hez ugrunk (mivel $find(g) = e$ ekkor) és frissítjük k_1 $OUT[e]$ -vel ($OUT[e] = 11$).
- Ekkor mivel $k_1 < k_2$ nem teljesül ezért leáll a while ciklus és vége a bejárásnak.



Tétel:

Adott egy irányítatlan, súlyozott $G = (V, E)$ gráf minimális feszítőfája, és a nem feszítőfabeli élek súly szerint rendezve, akkor az algoritmus $\mathcal{O}(m + n)$ időben és $\mathcal{O}(m + n)$ helyigény mellett megkeresni G minimális feszítőfájának összes minimális költségű csereélét.

Bizonyítás:

Legyen T a G gráf minimális feszítőfája.

- 1. A P szülő tömb inicializálása: $\mathcal{O}(n)$
- 2. Mivel T -ben $n - 1$ él van, ezért a DFS lefuttatása T -n az IN és OUT értékek inicializálásához: $\mathcal{O}(n)$.
- 3. Csereélek inicializálása minden MST élhez: $\mathcal{O}(n)$ idő.

3. Állítás:

Az algoritmus csak azokat az éleket járja be, amelyek benne vannak a nem-MST él által indukált körben.

- 4. $m - n + 1 = \mathcal{O}(m)$ nem-MST él olvasás van, növekvő sorban súly szerint: $\mathcal{O}(m)$ időkomplexitás.
- 5. Minden nem-MST élhez esetében a 3. állítás szerint, az algoritmus kizárólag az adott él által meghatározott fundamentális kör éleire hivatkozhat.

Időkomplexitás bizonyítása (3.)

- Ezeket az éleket csak egyszer járjuk be a következő módon:
 - 1 Az algoritmus minden fundamentális kört a leszármazottól az ősz irányába járja be. (a DSF bejárás közben létrehozott IN és OUT tömb értékei alapján)
 - 2 On visiting a vertex v , if v 's set label equals v then the edge $(v, P[v])$ has not been visited before, otherwise it violates the path compression at lines 13-15.
 - 3 When an edge $(v, P[v])$ gets a replacement (line 14), the disjoint set corresponding with parent $P[v]$ is united with v 's set (line 15) using the Gabow-Tarjan disjoint set link operation.
 - 4 The label of the new set is the root of the induced subtree of the MST. Therefore when a vertex v is first visited, $\text{link}(v)$ results in the set label being the label of the set containing $P[v]$.
 - 5 On completing the walk along the cycle, the set label will be the label of the set containing the LCA.
 - 6 Then subsequent $\text{find}(v)$ operations return the most recent root of the subtree containing v .
 - 7 Thus this sequence of disjoint set unions perform path compression on tree edges with assigned replacements. The compressed path decreases the traversal length of subsequent walks beginning at vertices lower in the DFS ordering by skipping over tree edges already with assigned replacement edges.

4. Állítás:

Az algoritmus $\mathcal{O}(m+n)$ időben és $\mathcal{O}(n)$ tárhellyel frissíti a diszjunkt halmazokat.

- 6. It follows from Claim 3 and this specific ordering of the disjoint set labels that the algorithm cannot follow a path that does not close the cycle.
- 7. Aztán az útkompresszió miatt csak $\mathcal{O}(m)$ élet járunk be, ami $\mathcal{O}(m)$ idő.
- 8. A 4. állítás szerint az összes művelete a diszjunkt halmazokon $\mathcal{O}(m+n)$ időkomplexitással és $\mathcal{O}(n)$ tárhelykomplexitással jár. Ezért $\mathcal{O}(m+n)$ időkomplexitással találjuk meg a csereéleket T-hez.
- 9. A használt adatszerkezetek: egyszerű tömbök és Gabow-Tarjan diszjunkt halmaz adatszerkezetek, amik $\mathcal{O}(n)$ tárhely. Továbbá, az összes nem-MST él $\mathcal{O}(m)$ tárhely. Így igaz az előzőekben állított $\mathcal{O}(m+n)$ időkomplexitás és $\mathcal{O}(m+n)$ tárhelykomplexitás. $\square$

Definíció:

Egy összefüggő, súlyozott G gráf legfontosabb éle, az az él, amelynek eltávolítása a legnagyobb növekedést eredményezi a minimális feszítőfa súlyában.

- Suraweera et al. bebizonyították, hogy a legfontosabb él a minimális feszítőfában van.
- **Így az algoritmus eredménye:**
 - Az összes csereél meghatározása után a legfontosabb él kiválasztható $\mathcal{O}(n)$ időben. Ehhez csak meg kell keresni, hogy melyik MST-beli él súlyának a különbsége legnagyobb a csereélének súlyával.
 - Ezzel: a **legfontosabb él kiszámítható $\mathcal{O}(m+n)$ időben**, ha adott az eredeti MST és a nem MST élek rendezve vannak.

- Képesek vagyunk egy minimális feszítőfa minden élének a csereélét megtalálni $\mathcal{O}(m + n)$ időben, ha a nem MST éleket növekvő sorrendben kapjuk meg.
- Ehhez Gabow és Tarjan speciális diszjunkt halmaz adatszerkezetét használjuk fel.
- Extra: az algoritmus a legfontosabb élet is képes megtalálni lineáris időben.

Köszönöm a figyelmet!